ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

STUDENT NAME: MUTANGANA Joseph

STUDENT ID: 29061

COURSE NAME: SOFTWARE QUALITY ASSURANCE

GROUP: C

REPORT: QUIZ 1

DATE: 05/07/2026

Question 1: Software Quality and its importance.

Software quality is the degree to which software satisfies requirements, user needs and standards.

Importance	Explanation	Example
Meets user requirements	Ensures software performs intended functions.	Student registration works correctly.
Reliability	Reduces failures.	Banking app processes transactions safely.
Customer satisfaction	Builds user confidence.	Reliable shopping app.
Lower maintenance cost	Fewer bugs to fix.	Early bug detection saves money.
Security	Protects data.	Encrypted login credentials.
Business success	Improves reputation.	Reliable university portal.

Example: A university registration system should register courses correctly, protect student data and remain available.

Question 2: The historical evolution of SQA.

SQA evolved from simple inspection to continuous quality throughout the SDLC.

Era	Period	Characteristics
Inspection & Testing	1940s-1960s	Quality mainly through inspection/testing.
Statistical Quality	1970s	Metrics and statistical control introduced.
Process Quality	1980s	TQM and ISO 9000 emphasized process.
CMM & Standards Era	1990s	CMM, IEEE and ISO/IEC 9126 adopted.
Agile & DevOps SQA Era	2000s-Present	Agile, DevOps and automation.

Example: Modern teams use automated testing and continuous quality instead of testing only at the end.

Question 3: Role and responsibilities of the (SQA) team.

The SQA team ensures that software is developed according to defined quality standards, processes, and customer requirements throughout the Software Development Life Cycle (SDLC).

Role	Responsibilities	Example
SQA Manager	Plans quality activities, defines quality policies, manages the SQA team.	Approves the project's quality plan.
SQA Engineer	Monitors development processes, reviews deliverables, ensures standards are followed.	Checks whether developers follow coding standards.
Test Engineer	Designs and executes test cases, reports defects.	Tests the login module and reports bugs.
Quality Auditor	Audits processes and verifies compliance with standards.	Verifies compliance with ISO procedures.
Configuration Manager	Controls versions and software releases.	Maintains version history using Git.
Project Manager	Ensures quality activities are integrated into the project.	Allocates time for testing before release.

Question 4: Importance of software quality standards and frameworks.
Software quality standards and frameworks provide internationally accepted guidelines for developing, evaluating, and improving software quality.

Standard/Framework	Purpose	Importance
ISO 9001	Quality management system	Improves organizational processes.
ISO/IEC 25010	Software quality model	Evaluates software quality characteristics.
IEEE Standards	Testing and documentation guidance	Promotes consistency in software engineering.
CMMI	Process improvement framework	Increases process maturity.
Six Sigma	Reduces defects using data	Improves product quality.

Example: A software company adopting ISO/IEC 25010 evaluates functionality, reliability, usability, security and maintainability before releasing its student management system.

Question 5: Compare the FURPS Quality Model and the ISO/IEC 25010 Quality Model
Both FURPS and ISO/IEC 25010 are software quality models used to evaluate software products. FURPS is simpler and focuses on five major quality categories, while ISO/IEC 25010 is the current international standard providing broader coverage for modern software systems.

Aspect	FURPS	ISO/IEC 25010
Main focus	5 quality categories	8 product quality characteristics + 5 quality-in-use characteristics
Characteristics	Functionality, Usability, Reliability, Performance, Supportability	Functional suitability, Performance efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability
Security	Included indirectly	Separate top-level characteristic
Compatibility	Not explicit	Explicit compatibility characteristic
Standard	HP model	International ISO standard
Best suited	Agile/startups, lightweight projects	Enterprise, government, safety-critical and modern systems

Question 6: the relationship between Quality Factors, Quality Criteria, and Quality Metrics
Software quality is evaluated through a hierarchy consisting of Quality Factors, Quality Criteria and Quality Metrics.

Level	Meaning	Example	Measurement
Quality Factor	High-level quality goal	Reliability	Not measured directly
Quality Criteria	Attributes contributing to the factor	Fault tolerance, Recoverability	Evaluated through observations/tests
Quality Metric	Numerical measurement	MTBF, Defect density	Actual numeric values

QUESTION 7:

The Spiral Model is a risk-driven Software Development Life Cycle (SDLC) model proposed by Barry Boehm. It combines elements of the Waterfall model and prototyping while emphasizing continuous risk analysis and customer feedback. Each cycle (spiral) results in a more complete version of the software until the final product is delivered.

Phases of the Spiral Model

Phase	Description	Quality Assurance Activities
Planning	Define objectives, requirements, and constraints.	Verify requirements through reviews and stakeholder validation.
Risk Analysis	Identify and evaluate technical and project risks.	Conduct risk assessment, feasibility studies, and quality planning.
Engineering	Design, code, and test the software.	Code reviews, unit testing, integration testing, and inspections.
Evaluation	Customer evaluates the developed version and provides feedback.	User Acceptance Testing (UAT), defect reporting, and process improvement.

Example

A **mobile banking application** can be developed using the Spiral Model.

- During the first iteration, developers create the login module.
- QA verifies security requirements and performs authentication testing.
- In the next iteration, developers add money transfer features.
- QA conducts functional testing, performance testing, and security testing before customer evaluation.
- Each iteration improves software quality until the complete system is delivered.

QUESTION 8: Explain the Key Principles of DevOps and Their Contribution to Software Quality

DevOps is a software development approach that combines Development (Dev**)** and Operations (Ops**)** to improve collaboration, automation, and continuous software delivery. DevOps enables organizations to deliver high-quality software faster while maintaining system reliability.

Key Principles of DevOps

Principle	Description	Contribution to Software Quality
Collaboration	Developers and operations teams work together.	Reduces communication errors and improves productivity.
Continuous Integration (CI)	Developers frequently merge code into a shared repository.	Detects integration errors early.
Continuous Delivery (CD)	Software is automatically prepared for deployment.	Faster and more reliable software releases.
Automation	Automates building, testing, and deployment processes.	Reduces human errors and improves consistency.
Continuous Monitoring	Continuously monitors applications and infrastructure.	Detects issues before users are affected.
Feedback	Continuous customer and operational feedback.	Supports continuous improvement and better user satisfaction.

Example

Netflix uses DevOps practices to deploy software updates many times every day.

Through automated testing, continuous integration, and continuous monitoring, Netflix quickly identifies and resolves issues while maintaining high service availability for millions of users worldwide.

QUESTION 9: Importance of Quality Gates in a CI/CD Pipeline

A **Quality Gate** is a checkpoint in a Continuous Integration/Continuous Delivery (CI/CD) pipeline that determines whether software meets predefined quality standards before moving to the next stage. Quality gates help prevent defective code from reaching production by enforcing automated quality checks throughout the development lifecycle.

Importance of Quality Gates

Quality Gate	Purpose	Benefit
Build Validation	Confirms the application compiles successfully.	Prevents broken builds.
Automated Testing	Executes unit, integration, and system tests.	Detects defects early.
Code Quality Analysis	Checks coding standards, code smells, and security vulnerabilities.	Improves maintainability and security.
Deployment Approval	Confirms the software is ready for release.	Reduces deployment failures.

Example

A company developing an online shopping application uses GitHub Actions and SonarQube in its CI/CD pipeline.

Before deployment:

- The project must compile successfully.
- Unit tests must achieve at least **90% pass rate**.
- No critical security vulnerabilities are allowed.
- Code coverage must exceed **80%**.

If any condition fails, deployment stops automatically until the issue is corrected.

QUESTION 10: Activities Performed During the Requirements and Design Phases of Quality Assurance

Quality Assurance begins at the earliest stages of software development. During the Requirements and Design phases, QA focuses on preventing defects by reviewing requirements and validating system designs before coding begins.

Activities During the Requirements Phase

Activity	Description
Requirements Review	Ensures all requirements are complete, clear, and consistent.
Requirement Validation	Confirms requirements meet business and user needs.
Requirement Traceability	Links requirements to design, development, and testing activities.
Risk Analysis	Identifies risks associated with incomplete or ambiguous requirements.
Feasibility Assessment	Evaluates whether requirements can realistically be implemented.

Activities During the Design Phase

Activity	Description
Design Review	Verifies that the system design satisfies all requirements.
Architecture Evaluation	Assesses scalability, security, maintainability, and performance.
Interface Review	Checks communication between software modules.
Database Design Review	Ensures efficient and reliable data storage structures.
Testability Review	Confirms the design supports effective testing.

QUESTION 11: Seven Principles of Software Testing

The Seven Principles of Software Testing

Principle	Explanation	Example
1. Testing shows the presence of defects, not their absence	Testing can reveal defects but cannot prove that software is completely error-free.	A payroll system may pass all tests but still contain an undiscovered bug.
2. Exhaustive testing is impossible	Testing every possible input and scenario is impractical because of time and cost constraints.	An ATM application with thousands of possible transactions cannot be tested exhaustively.
3. Early testing saves time and cost	Testing activities should begin during the requirements and design phases to detect defects early.	Reviewing requirements before coding prevents expensive redesign later.
4. Defect clustering	A small number of modules usually contain the majority of software defects.	Most defects in an e-commerce system may occur in the payment module.
5. Pesticide paradox	Repeating the same test cases eventually finds fewer new defects; tests should be updated regularly.	Adding new test cases after each software update increases defect detection.
6. Testing is context dependent	Different systems require different testing strategies depending on their purpose and risk level.	A banking application requires more security testing than a social media application.
7. Absence-of-errors fallacy	Software with no defects can still fail if it does not meet user requirements.	A bug-free system that lacks required features will still be rejected by users.

QUESTION 12: Differences Between Unit Testing, Integration Testing, System Testing, and Acceptance Testing

Software testing is performed at different levels to verify that each component of the software functions correctly before deployment. The four main testing levels are Unit Testing, Integration Testing, System Testing, and Acceptance Testing**.** Each level has a different objective, scope, and responsibility.

Key Differences

Feature	Unit Testing	Integration Testing	System Testing	Acceptance Testing
Scope	Single component	Multiple components	Entire system	Business requirements
Objective	Verify code correctness	Verify module interaction	Verify complete functionality	Verify user satisfaction
Environment	Development	Test environment	QA environment	Customer/User environment
Main Participants	Developers	Developers & Testers	QA Engineers	Clients/Users

QUESTION 13: Importance of Performance Testing and Differentiate Between Load Testing and Stress Testing

Performance Testing is a non-functional software testing technique used to determine how a software application performs under different workload conditions. It helps ensure that the software is fast, stable, responsive, and reliable before deployment.

Performance testing is important because users expect applications to respond quickly, even when many people use them simultaneously. Poor performance can lead to customer dissatisfaction, financial losses, and damage to an organization's reputation.

Importance of Performance Testing

Importance	Explanation
Improves system speed	Ensures the application responds quickly to user requests.
Measures scalability	Determines whether the application can support increasing numbers of users.
Identifies bottlenecks	Detects slow components such as databases, servers, or network connections.
Improves reliability	Ensures the application remains stable during heavy usage.
Enhances user satisfaction	Fast and responsive applications improve user experience.
Reduces business risks	Prevents failures during peak business periods.

Difference Between Load Testing and Stress Testing

Feature	Load Testing	Stress Testing
Purpose	Measures performance under expected user load.	Determines system behavior beyond its maximum capacity.
Objective	Verify normal performance.	Identify the system's breaking point.
Workload	Expected or average workload.	Extremely high workload.
Expected Result	System should remain stable.	System may fail but should recover gracefully.
Main Focus	Performance and response time.	Stability and recovery after failure.

QUESTION 14: Purpose and Importance of Test Documentation

Test documentation refers to all documents created before, during, and after software testing. These documents provide a structured approach to planning, executing, monitoring, and reporting testing activities.

Proper documentation improves communication among team members, supports project management, and ensures software quality throughout the development process.

Importance of Test Documentation

Benefit	Explanation
Improves communication	Ensures all stakeholders understand testing activities.
Supports traceability	Links requirements with corresponding test cases.
Facilitates maintenance	Makes future testing easier when software changes.
Promotes consistency	Ensures testing follows standardized procedures.
Saves time	Existing documents can be reused in future releases.
Assists auditing	Provides evidence that testing activities were completed.

QUESTION 15: Importance of Test Planning in Software Testing

Test planning is one of the most important activities in software testing. It defines the testing objectives, scope, resources, schedule, responsibilities, and testing strategy before testing begins. A well-prepared test plan ensures that testing activities are organized, efficient, and aligned with project requirements.

Importance of Test Planning

Importance	Explanation
Defines Testing Scope	Identifies which features will and will not be tested.
Efficient Resource Allocation	Assigns testers, tools, and resources appropriately.
Improves Time Management	Establishes a realistic testing schedule and milestones.
Reduces Project Risks	Identifies potential testing risks and mitigation strategies early.
Enhances Communication	Ensures all stakeholders understand testing objectives and responsibilities.
Improves Software Quality	Provides a structured approach that increases testing effectiveness.

QUESTION 16: Discuss the Role of Regression Testing in Agile Software Development

Regression testing is the process of re-testing software after changes have been made to ensure that existing functionality continues to work correctly. In Agile software development, where software is developed in short iterations and frequent changes are introduced, regression testing plays a critical role in maintaining software quality.

Real-World Example

An **e-commerce website** introduces a new online payment method.

Although the payment feature works correctly, regression testing is performed to ensure that:

- User login still functions correctly.
- Shopping cart operations remain unaffected.
- Order processing works normally.
- Existing payment methods continue to operate without errors.

Regression testing confirms that the new feature has not broken existing functionality.

QUESTION 17: Explain the Different Types of Test Environments Used During Software Testing

A **test environment** is a setup that provides the necessary hardware, software, network, databases, and tools required to execute software tests. Different test environments are used throughout the Software Development Life Cycle (SDLC) to ensure that software functions correctly before it is released to users.

Types of Test Environments

Test Environment	Purpose	Characteristics	Example
Development Environment	Used by developers to write and test code.	Frequent code changes, debugging tools available.	Developers testing a login module.
Testing (QA) Environment	Used by QA engineers to verify software functionality.	Stable environment with prepared test cases and test data.	Functional and regression testing.
Integration Environment	Verifies interaction between different software modules or systems.	Multiple components connected for testing.	Payment system communicating with a bank API.
User Acceptance Testing (UAT) Environment	Allows end users to validate the software before deployment.	Closely resembles the production environment.	Customers testing a university registration system.
Staging Environment	Final environment before production deployment.	Almost identical to production configuration.	Final verification before software release.
Production Environment	Live environment where end users access the software.	Real users and real data.	Online banking or e-commerce website.

QUESTION 18: Discuss the Importance of Test Automation in Modern Software Development

Test automation is the use of software tools and scripts to execute tests automatically instead of performing them manually. It has become an essential practice in modern software development because organizations release software more frequently and require faster, more reliable testing.

Importance of Test Automation

Benefit	Explanation
Faster Testing	Automated tests execute much faster than manual tests.
Improved Accuracy	Eliminates human errors during repetitive testing.
Supports Continuous Integration (CI)	Tests run automatically whenever code changes are made.
Better Regression Testing	Ensures existing features continue working after modifications.
Cost Savings	Reduces long-term testing costs for large projects.
Increased Test Coverage	Thousands of test cases can be executed automatically.

QUESTION 19: Explain the Challenges Commonly Faced in Software Quality Assurance and Suggest Possible Solutions

Software Quality Assurance (SQA) ensures that software products meet specified quality standards throughout the Software Development Life Cycle (SDLC). However, QA teams often face challenges that can affect software quality, project timelines, and customer satisfaction. Understanding these challenges and implementing appropriate solutions helps organizations produce reliable and high-quality software.

Common Challenges and Their Solutions

Challenge	Description	Possible Solution
Changing Requirements	Frequent requirement changes may introduce defects and increase testing effort.	Adopt Agile methodologies, maintain clear documentation, and perform continuous requirement reviews.
Limited Testing Time	Tight project deadlines reduce the time available for comprehensive testing.	Prioritize high-risk features, automate repetitive tests, and adopt continuous testing.
Inadequate Communication	Poor communication between developers, testers, and stakeholders leads to misunderstandings.	Hold regular meetings, use collaboration tools, and maintain transparent documentation.
Lack of Skilled Testers	Insufficient technical knowledge may reduce testing effectiveness.	Provide regular training, certifications, and knowledge-sharing sessions.
Budget Constraints	Limited financial resources restrict testing tools and personnel.	Focus on risk-based testing and utilize cost-effective open-source testing tools.
Inadequate Test Environment	Differences between test and production environments may produce inaccurate results.	Create realistic test environments and use virtualization or cloud-based testing platforms.
Poor Test Data Management	Inaccurate or insufficient test data may affect testing outcomes.	Develop structured test datasets and use automated test data generation tools.
Increasing Software Complexity	Modern software systems involve multiple technologies, APIs, and platforms.	Implement automated testing, continuous integration, and modular testing strategies.

QUESTION 20: Discuss How Effective Software Quality Assurance Contributes to the Successful Delivery of Software Projects

Software Quality Assurance (SQA) is a systematic process that ensures software products meet customer requirements, industry standards, and organizational objectives. Effective SQA contributes significantly to the successful delivery of software projects by improving product quality, reducing risks, and increasing customer satisfaction.

Contributions of Effective Software Quality Assurance

Contribution	Explanation
Improved Software Quality	Early defect detection produces reliable and stable software.
Reduced Development Cost	Identifying defects early minimizes the cost of correcting errors later in development.
Increased Customer Satisfaction	High-quality software meets user expectations and improves customer confidence.
Reduced Project Risks	Continuous testing identifies potential problems before deployment.
Improved Team Productivity	Clearly defined quality processes reduce rework and improve collaboration.
Compliance with Standards	Ensures adherence to organizational, legal, and international quality standards such as ISO/IEC 25010.
Faster Software Delivery	Automated testing and continuous integration accelerate software releases.
Better Maintainability	Well-tested software is easier to update, maintain, and enhance.

Example

An e-commerce company developing an online shopping platform integrates Software Quality Assurance throughout the Software Development Life Cycle.

The QA team performs:

- Requirement reviews.
- Automated unit and integration testing.
- Performance and security testing.
- User Acceptance Testing (UAT).

As a result:

- Software defects are detected early.
- System downtime is minimized.
- Customer satisfaction increases.
- The project is completed on schedule and within budget.